

OGX: An Open-Source, Vendor-Neutral Generative AI Application Server

Francisco Javier Arceo¹, Sébastien Han¹, Matthew Farrellee³, Charlie Doern¹, Yuan Tang¹, Derek Higgins¹, Varsha Prasad Narsing¹, Gordon Sim¹, Sumanth Kamenani¹, Ben Browning¹, and Raghotham Murthy²

¹ Red Hat AI, USA ² Meta, USA ³ Independent [✉] Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Fangzhou Xie](#)

Reviewers:

- [@showdawn](#)
- [@alexzerg](#)

Submitted: 05 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

OGX (Open GenAI Stack), formerly Llama Stack ([Llama Stack Contributors, 2025](#)), is an open-source AI application server and Python library with interchangeable backend providers ([OGX Contributors, 2026b](#)). Teams building retrieval-augmented generation (RAG) pipelines, conversational agents, and tool-calling workflows can change their inference engine, vector database, and safety backend without rewriting application code.

OGX implements OpenAI, Anthropic, and Google APIs, with the Responses API for server-side orchestration as its primary focus ([Open Responses Community, 2026](#)). It supports over 20 inference providers, 13 vector store backends, and 7 safety providers. A companion Kubernetes Operator ([OGX Contributors, 2026a](#)) manages deployments, including shared and per-tenant instances. Together, OGX and its operator provide a self-hosted backend for AI-powered developer tools such as Claude Code, Codex CLI, OpenCode, and OpenHands.

Statement of Need

AI application development today is tightly coupled to proprietary API providers. While inference-only workloads can increasingly be swapped across providers—vLLM, for example, supports the Responses API for basic inference—applications that rely on the full stack (retrieval, tool calling, conversation state, safety guardrails) remain difficult to migrate without rewriting significant application logic. This coupling limits reproducibility, makes comparisons across model providers difficult, and prevents teams from running AI workloads on controlled infrastructure—a requirement in regulated, privacy-sensitive, and air-gapped environments.

OGX addresses this need through standard APIs and configurable infrastructure. Researchers can compare backends while keeping retrieval, tool authorization, and conversation handling in one server. This separates the choice of client SDK, model, and deployment environment, supporting reproducible configurations on controlled infrastructure.

State of the Field

OGX continues Llama Stack under a renamed, model-agnostic mission. Its server-side API layer complements inference engines, gateways, and client-side frameworks, which address different parts of the deployment problem.

Inference engines (vLLM ([Kwon et al., 2023](#)), SGLang ([Zheng et al., 2024](#)), Ollama) focus on efficient model serving. They optimize throughput and latency but do not provide retrieval, conversation state, tool execution, or safety guardrails. An application using vLLM for inference

38 must separately integrate a vector database, implement its own agentic loop, and manage
39 multi-turn state.

40 **API gateways** (LiteLLM, OpenRouter) provide a unified interface across multiple inference
41 providers but act as pass-through proxies. They do not manage vector stores, execute tool
42 calls, or maintain conversation history—they translate request formats between SDKs and
43 providers.

44 **Client-side frameworks** (LangChain (LangChain, Inc., 2023), LangGraph (LangChain, Inc.,
45 2024), LlamaIndex (LlamaIndex, 2022), CrewAI (CrewAI, Inc., 2024), Haystack (deepset,
46 2023)) provide rich developer abstractions for building agents and RAG pipelines. However,
47 they execute orchestration client-side, distributing security-critical logic across application code.
48 These frameworks are complementary to OGX: they compose agent logic while OGX provides
49 the server-side execution target they call into.

50 **Proprietary platforms** (OpenAI's Responses API (OpenAI, 2025), Databricks Mosaic AI
51 (Databricks, 2025)) offer integrated experiences but couple applications to a specific vendor's
52 infrastructure and pricing.

53 OGX integrates inference, retrieval, tool execution, conversation management, and safety
54 behind Open Responses-compatible APIs (Open Responses Community, 2026). Combining
55 these responsibilities in a server addresses a different trust boundary from extending a client-side
56 framework: applications share centralized policy enforcement and provider configuration while
57 retaining their own agent logic.

58 Software Design

59 Provider Architecture

60 OGX's core abstraction is the pluggable provider. Each API capability (inference, vector storage,
61 safety, tool runtime, file processing) is defined by a Protocol interface in the lightweight `ogx-`
62 `api` package, allowing third-party providers to implement the contract without depending
63 on the full server. Concrete providers implement these interfaces for specific backends:
64 `remote::openai` and `remote::anthropic` for hosted APIs, `remote::vllm` for self-hosted GPU
65 inference, `inline::faiss` and `remote::pgvector` for vector search, and so on. A routing layer
66 dispatches requests to provider instances based on logical resource identifiers, enabling multiple
67 providers to serve the same API simultaneously—for example, Ollama handling local models
68 while OpenAI handles hosted models, both behind `/v1/chat/completions`.

69 A *distribution* packages a specific set of providers and configuration into a deployable unit,
70 decoupling application logic from infrastructure selection. This design favors reproducible
71 configuration over ad hoc application code: teams can prototype with lightweight inline
72 providers (Ollama, `sqlite-vec` (Garcia, 2024)) and deploy to production backends (vLLM,
73 `pgvector`) by changing the distribution, not the application.

74 Dual Deployment Model

75 OGX runs in two modes. **Server mode** exposes HTTP endpoints accessible from any language
76 or tool. **Library mode** allows direct Python import with zero network overhead, suitable for
77 notebooks and scripts. Both modes use identical provider routing and API semantics.

78 Multi-SDK Compatibility

79 OGX serves three client SDK protocols from a single server. The **OpenAI-compatible endpoints**
80 (`/v1/chat/completions`, `/v1/responses`, `/v1/vector_stores`) are the primary interface and
81 the Responses API implementation conforms to the Open Responses specification (Open
82 Responses Community, 2026). The **Anthropic Messages endpoint** (`/v1/messages`) and

83 **Google GenAI Interactions endpoint** (`/v1alpha/interactions`) provide native compatibility
84 for teams using those SDKs. This portability comes with a trade-off: OGX must normalize
85 provider-specific behavior into stable API contracts while still exposing enough backend-specific
86 configuration for real deployments.

87 **Server-Side Agentic Orchestration**

88 The Responses API implements server-side agentic orchestration: the inference-tool-inference
89 loop executes within the server process, not the client ([OpenAI, 2025](#)). This centralizes security
90 enforcement, tool authorization, and conversation state management, at the cost of moving
91 some flexibility from application code into server configuration. Built-in tools include file search
92 (RAG over vector stores), web search, code interpretation, and Model Context Protocol (MCP)
93 ([Anthropic, 2026](#)) integration for external tool servers.

94 OGX is extending server-side execution with a Containers API and a Skills API. The Containers
95 API (`/v1/containers`) manages sandboxed execution environments—matching the OpenAI
96 Containers API—enabling models to execute shell commands in isolated Docker, Podman, or
97 Kubernetes containers via a shell tool in the Responses API. The Skills API (`/v1alpha/skills`)
98 manages versioned skill bundles that package tools, prompts, and configuration into reusable,
99 composable units. Together, these APIs move code execution and task composition onto the
100 server, extending the same trust-boundary principle that governs retrieval and tool authorization.

101 Unlike inference engines and API gateways that treat requests as stateless, OGX manages
102 state natively: the Conversations API persists multi-turn history with tenant-scoped isolation,
103 the Prompts API provides versioned prompt templates, and a resource registry tracks models,
104 vector stores, and files as first-class server objects. A Compaction API summarizes long histories
105 to manage context window limits. Telemetry is built on OpenTelemetry (OTEL) with MLflow
106 ([Zaharia et al., 2018](#)) tracing integration for logging spans, tool calls, and retrieval steps to
107 existing ML experiment tracking infrastructure. This state and observability layer is what
108 makes OGX a complete application server rather than a stateless proxy.

109 For multitenant deployments, OGX provides attribute-based access control (ABAC) that
110 enforces tenant isolation at the retrieval, tool execution, state management, and API routing
111 layers. The security properties of this architecture have been formally analyzed and empirically
112 validated ([Arceo & Narsing, 2026](#)).

113 **Kubernetes Operator**

114 The OGX Kubernetes Operator ([OGX Contributors, 2026a](#)) provides declarative deployment
115 through the OGXServer custom resource. A single CR specifies the distribution, replica
116 count, storage, inference backend, and network policies. The operator manages full lifecycle
117 reconciliation with support for both vanilla Kubernetes and OpenShift, ConfigMap-driven image
118 overrides for fleet-wide updates, multi-architecture builds (amd64/arm64) with FIPS-compliant
119 images, and shared instances with ABAC isolation, per-tenant namespace isolation, and hybrid
120 topologies.

121 **Example Usage**

122 This RAG example uses the standard OpenAI SDK against OGX, with inference and vector
123 storage selected through server configuration:

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8321/v1", api_key="unused")

# Create a vector store and upload documents
```

```
vector_store = client.vector_stores.create(name="docs")
with open("manual.pdf", "rb") as file:
    client.vector_stores.files.upload(vector_store_id=vector_store.id, file=file)

# Query with server-side RAG via the Responses API
response = client.responses.create(
    model="meta-llama/Llama-3.2-3B-Instruct",
    input="What are the installation requirements?",
    tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]},
)
print(response.output_text)
```

Switching from Ollama to vLLM changes the server configuration while preserving this client code. Published tutorials cover IBM watsonx.ai with Milvus ([IBM Community, 2025](#)) and Oracle Cloud Infrastructure with OCI AI Blueprints ([Oracle, 2025](#)).

Research Impact Statement

OGX has realized impact through both public deployments and customer production use. Publicly documented examples under the former Llama Stack name include an intelligent OpenShift operations agent combining RAG, web search, and MCP tool integration for automated incident response ([Red Hat, 2025](#)), enterprise RAG pipelines on IBM watsonx.data with Milvus ([IBM Community, 2025](#)), and generative AI application development on Oracle Cloud Infrastructure ([Oracle, 2025](#)). Beyond these publicly documented cases, maintainers are also aware of additional production use by enterprise customers in sectors including telecommunications, semiconductor manufacturing, financial services, insurance, and consulting; these engagements are confidential and not independently documented, so we report them here as self-reported maintainer knowledge rather than citable evidence. The framework has been presented at Meta Connect ([Meta, 2024](#)) and IBM TechXchange ([Clyburn, 2025](#)) as a standardization layer for enterprise AI applications.

The security architecture—specifically, the multitenant isolation model combining ABAC-gated retrieval, server-side orchestration, and pluggable provider backends—was formally analyzed in a peer-reviewed publication at the ACM Conference on AI and Agentic Systems ([Arceo & Narsing, 2026](#)). OGX conforms to the Open Responses specification ([Open Responses Community, 2026](#)) and serves as a reference implementation for open, vendor-neutral agentic AI APIs.

As of June 2026, the project has over 8,400 GitHub stars, 242 contributors, 4,000 commits, and 68 releases across nearly two years of public development. Community engagement includes weekly contributor calls, an active Discord server, and integrations contributed by external organizations including Red Hat, IBM, Oracle, and Infinispan.

AI Usage Disclosure

Generative AI tools, including GitHub Copilot, Anthropic Claude, and OpenAI Codex (GPT-6), were used for code completion, documentation drafting, and paper drafting. Assistance was limited to generating candidate text or code that human contributors reviewed, edited, tested, and validated. Core architectural decisions, API design, the security model, and final paper content were made by human authors.

Acknowledgements

We thank Meta for creating and open-sourcing Llama Stack, now renamed OGX. We are grateful to Red Hat for supporting the development of OGX through employee time and infrastructure support. We thank the OGX contributor community for their sustained contributions to the project.

References

- Anthropic. (2026). *Model context protocol specification*. <https://modelcontextprotocol.io/specification/>
- Arceo, F. J., & Narsing, V. P. (2026). Securing the agent: Vendor-neutral, multitenant enterprise retrieval and tool use. *Proceedings of the ACM Conference on AI and Agentic Systems*, 862–872. <https://doi.org/10.1145/3786335.3813145>
- Clyburn, C. (2025). *Llama stack: Kubernetes for RAG and AI agents in generative AI*. https://mediacenter.ibm.com/media/Llama+Stack+Kubernetes+for+RAG+AI+Agents+in+Generative+AI/1_xl78upq2
- CrewAI, Inc. (2024). *CrewAI: Framework for orchestrating role-playing autonomous AI agents*. <https://github.com/crewAIInc/crewAI>
- Databricks. (2025). *Mosaic AI Agent Framework*. <https://www.databricks.com/product/machine-learning/retrieval-augmented-generation>
- deepset. (2023). *Haystack: End-to-end LLM framework for building production-ready applications*. <https://github.com/deepset-ai/haystack>
- Garcia, A. (2024). *Sqlite-vec: A vector search SQLite extension*. <https://github.com/asg017/sqlite-vec>
- IBM Community. (2025). *Build RAG with Llama Stack and watsonx.data Milvus*. <https://community.ibm.com/community/user/blogs/divya13/2025/05/08/build-rag-with-llama-stack-and-watsonxdata-milvus>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 611–626. <https://doi.org/10.1145/3600006.3613165>
- LangChain, Inc. (2023). *LangChain: Build context-aware reasoning applications*. <https://github.com/langchain-ai/langchain>
- LangChain, Inc. (2024). *LangGraph: Build resilient language agents as graphs*. <https://github.com/langchain-ai/langgraph>
- Llama Stack Contributors. (2025). *Llama stack*. <https://github.com/llamastack/llama-stack>
- LlamaIndex. (2022). *LlamaIndex: Data framework for LLM applications*. https://github.com/run-llama/llama_index
- Meta. (2024). *Llama stack: Chapter one*. <https://developers.facebook.com/m/meta-connect-developer-sessions/llama-stack-chapter-one/>
- OGX Contributors. (2026a). *OGX kubernetes operator*. <https://github.com/ogx-ai/ogx-k8s-operator/tree/7fa16532e1434bf74493ca305b1e21030914ae57>
- OGX Contributors. (2026b). *OGX (open GenAI stack)*. <https://github.com/ogx-ai/ogx/tree/5393c94b2d23a3069b3708f9ca81ad350d2deb21>
- Open Responses Community. (2026). *Open responses specification*. <https://www.>

199 openresponses.org/

200 OpenAI. (2025). *Responses API reference*. [https://platform.openai.com/docs/api-reference/](https://platform.openai.com/docs/api-reference/responses)
201 [responses](https://platform.openai.com/docs/api-reference/responses)

202 Oracle. (2025). *Accelerating enterprise gen AI applications development on OCI with Llama*
203 *Stack and OCI AI blueprints*. [https://blogs.oracle.com/ai-and-datascience/accelerating-](https://blogs.oracle.com/ai-and-datascience/accelerating-enterprise-gen-ai-applications-development-on-oci-with-llama-stack-and-oci-ai-blueprints)
204 [enterprise-gen-ai-applications-development-on-oci-with-llama-stack-and-oci-ai-blueprints](https://blogs.oracle.com/ai-and-datascience/accelerating-enterprise-gen-ai-applications-development-on-oci-with-llama-stack-and-oci-ai-blueprints)

205 Red Hat. (2025). *Generative AI applications with Llama Stack: A notebook-guided*
206 *journey to an intelligent operations agent*. [https://www.redhat.com/en/blog/generative-](https://www.redhat.com/en/blog/generative-ai-applications-llama-stack-notebook-guided-journey-intelligent-operations-agent)
207 [ai-applications-llama-stack-notebook-guided-journey-intelligent-operations-agent](https://www.redhat.com/en/blog/generative-ai-applications-llama-stack-notebook-guided-journey-intelligent-operations-agent)

208 Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching,
209 S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating
210 the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41, 39–45. [https:](https://api.semanticscholar.org/CorpusID:83459546)
211 [//api.semanticscholar.org/CorpusID:83459546](https://api.semanticscholar.org/CorpusID:83459546)

212 Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica,
213 I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient execution of
214 structured language model programs. *Advances in Neural Information Processing Systems*,
215 37. <https://doi.org/10.52202/079017-2000>